

Problem Statement

- Urban traffic systems lack intelligent, real-time monitoring.
- Manual control is inefficient for rapidly changing traffic conditions.
- Existing solutions are costly, infrastructure-heavy, and not scalable.
- Need for a compact, real-time, and low-cost prototype for intelligent traffic surveillance.

Objective/Purpose

- Simulate a smart traffic junction on a miniature scale.
- Use YOLOv8 to detect and count vehicles in real-time.
- Detect crashes using bounding box overlap (IoU logic).
- Send instant alerts via Telegram bot when crashes are detected.
- Use Jetson Nano for efficient on-device AI inference.

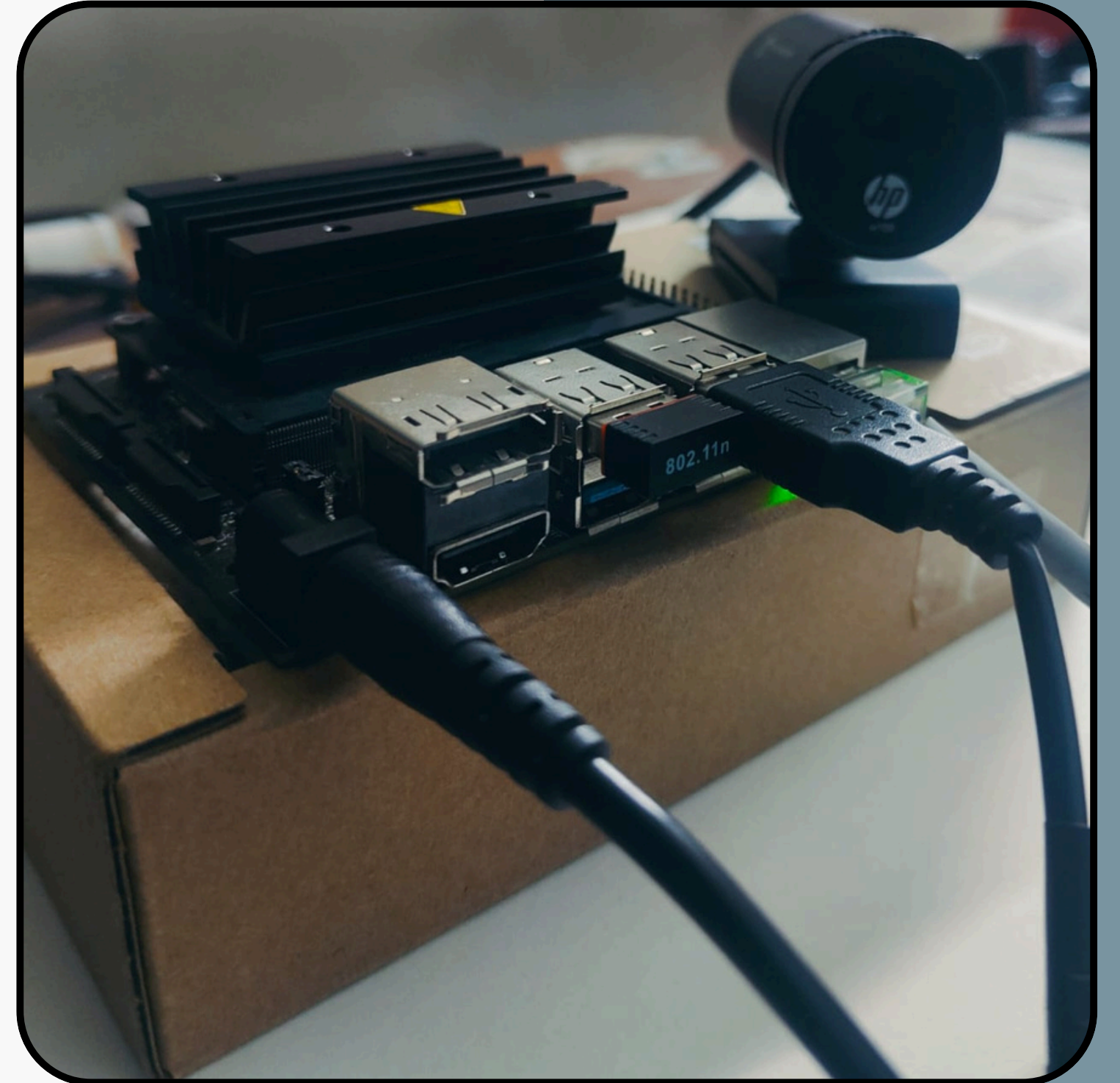
Project Requirements

Hardware:

- NVIDIA Jetson Nano (4GB RAM)
- USB Camera
- Thermocol base, black A4 sheets (for roads)
- Hot Wheels toy cars (as traffic)

Software & Libraries:

- Ubuntu 20.04
- Python 3.8+
- YOLOv8 (Ultralytics)
- OpenCV, Numpy
- Telegram Bot API



Literature Review



Traditional Methods:

- Background subtraction, contour analysis, motion detection
- Limitations: affected by lighting, noise, occlusion

YOLO Models:

- Single-shot detection architecture
- Real-time performance on edge devices
- YOLOv8 improves accuracy, speed, and robustness



Literature Review



Jetson Nano in Edge AI:

- Compact GPU-based platform for real-time processing
- Used in agriculture, security, and robotics

IoT Integration:

- Telegram Bot API offers lightweight real-time notifications
- Suitable for embedded and remote monitoring systems

Gap Addressed:

- Lack of miniature, embedded prototypes for traffic monitoring
- This project bridges the gap with integrated detection and alerting



Methodology Overview

1 Dataset Creation:

- Collected & labeled around 300 Hot Wheels images.
- Augmented to 1,500 images for better model performance.

2 Model Training:

- Trained YOLOv8 for 100 epochs on the custom dataset.

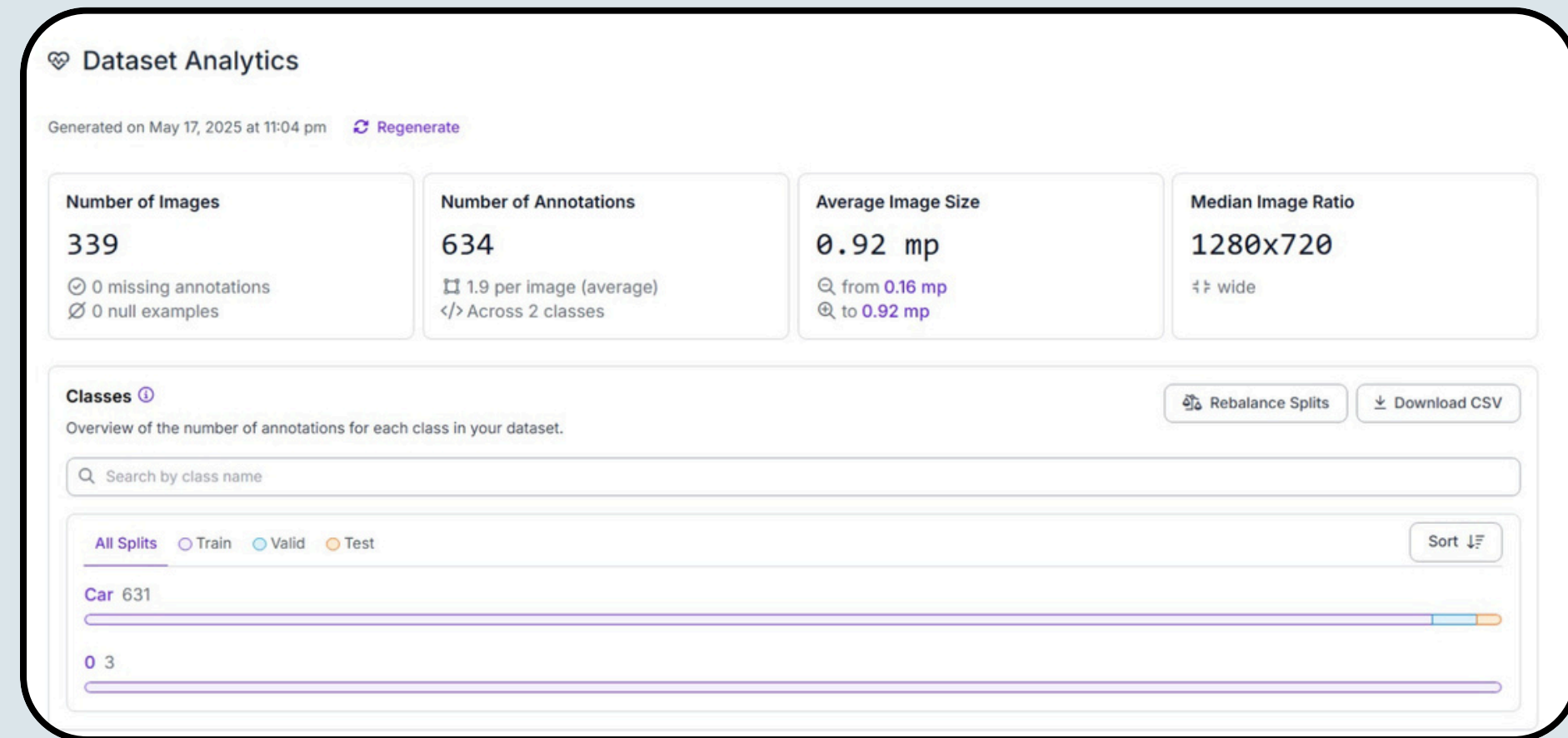
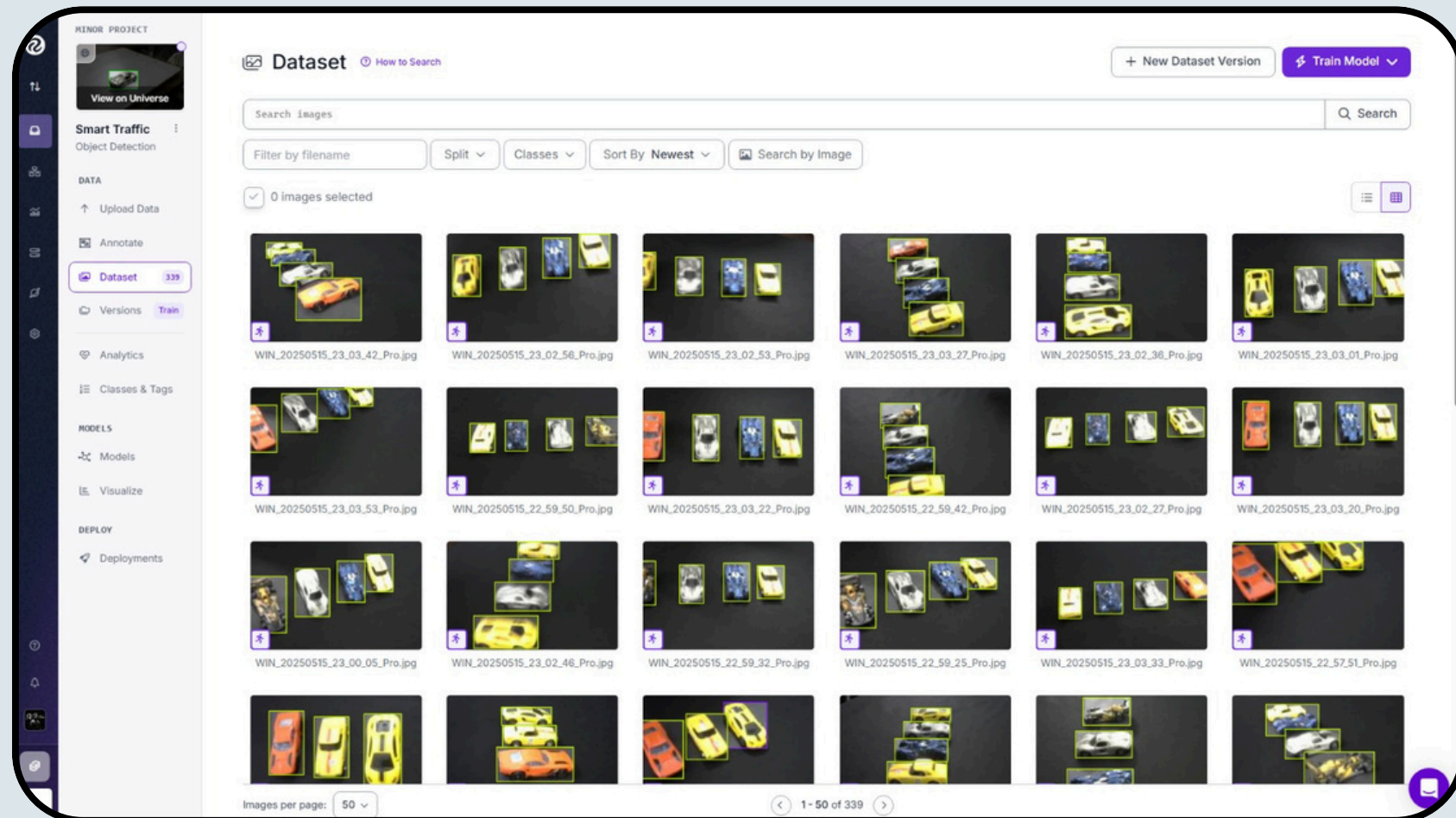
3 Deployment:

- Flashed Jetson with Ubuntu 20.04 and set up environment.
- Deployed trained model for real-time detection.

4 Detection & Alerts:

- Detected crashes and counted cars using video input.
- Sent instant Telegram notifications on crash detection.

Generating Custom Dataset



1. Capturing Images of the 6 Distinct Hot Wheels Toy Cars
2. Define Bounding Boxes
3. Annotate/Label the defined boxes(car)

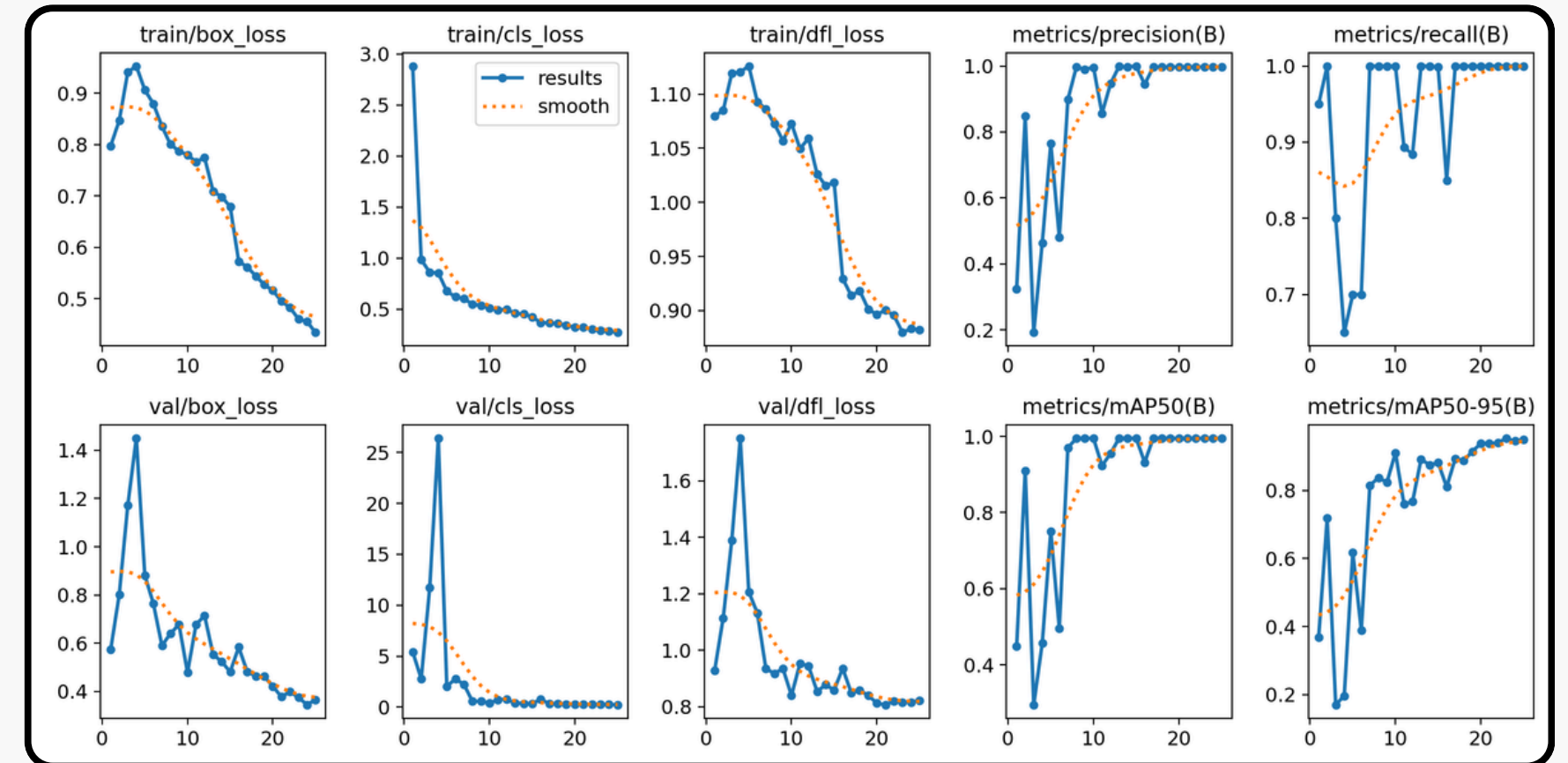
Model Training

- Total Images: ~300
- Augmentations: Rotation, flip, brightness
- Training Tool: Roboflow + Google Colab
- Model Used: YOLOv8n (lightweight, fast)
- Accuracy Achieved:

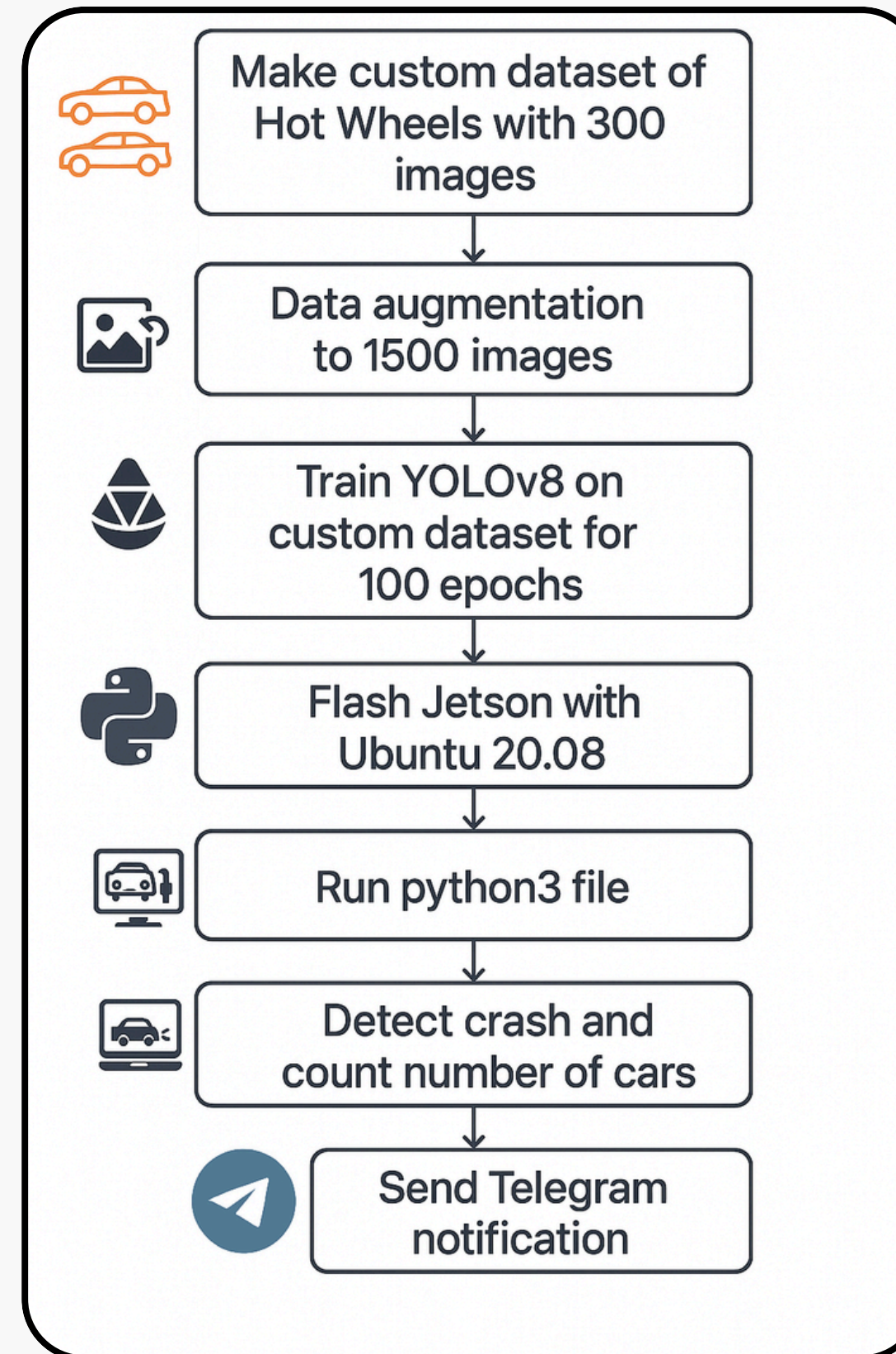
1. $mAP@0.5 \approx 85\%$

2. Precision $\approx 88\%$

3. Recall $\approx 86\%$



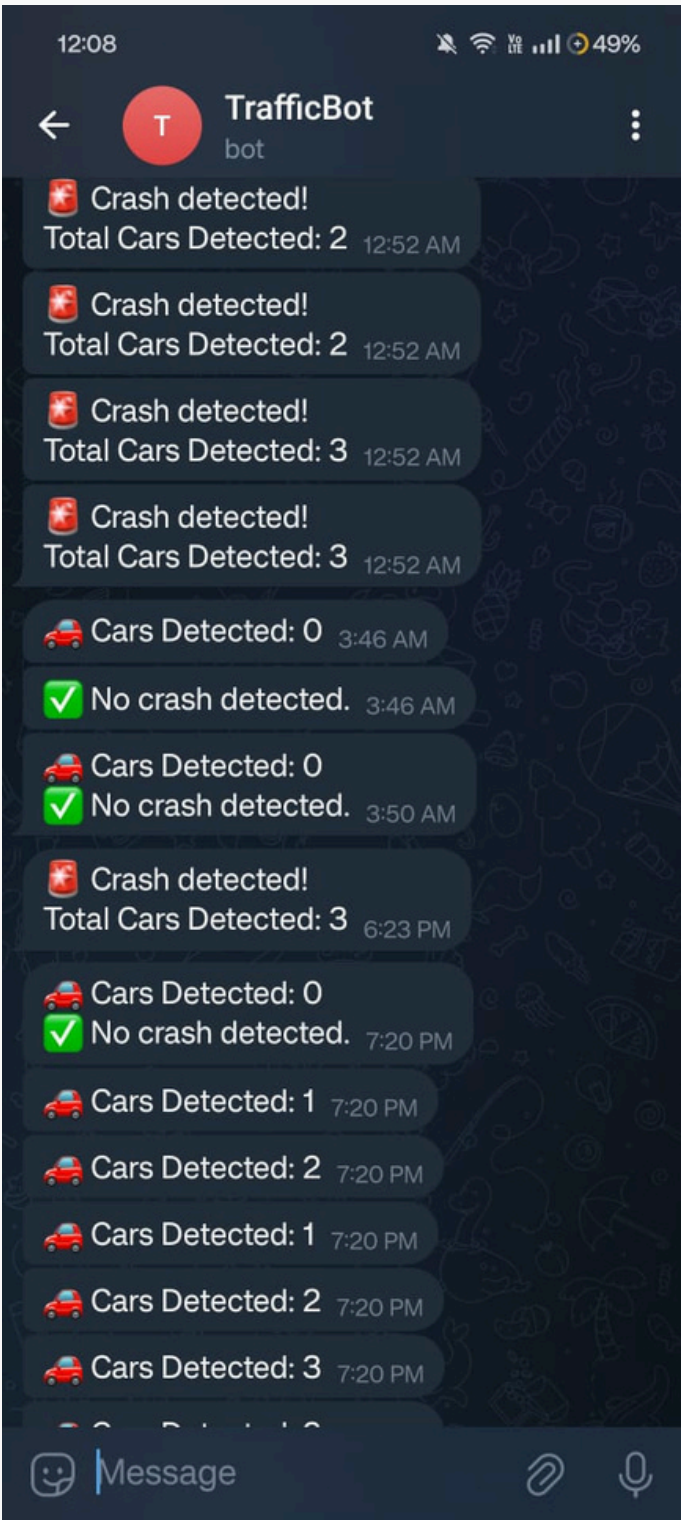
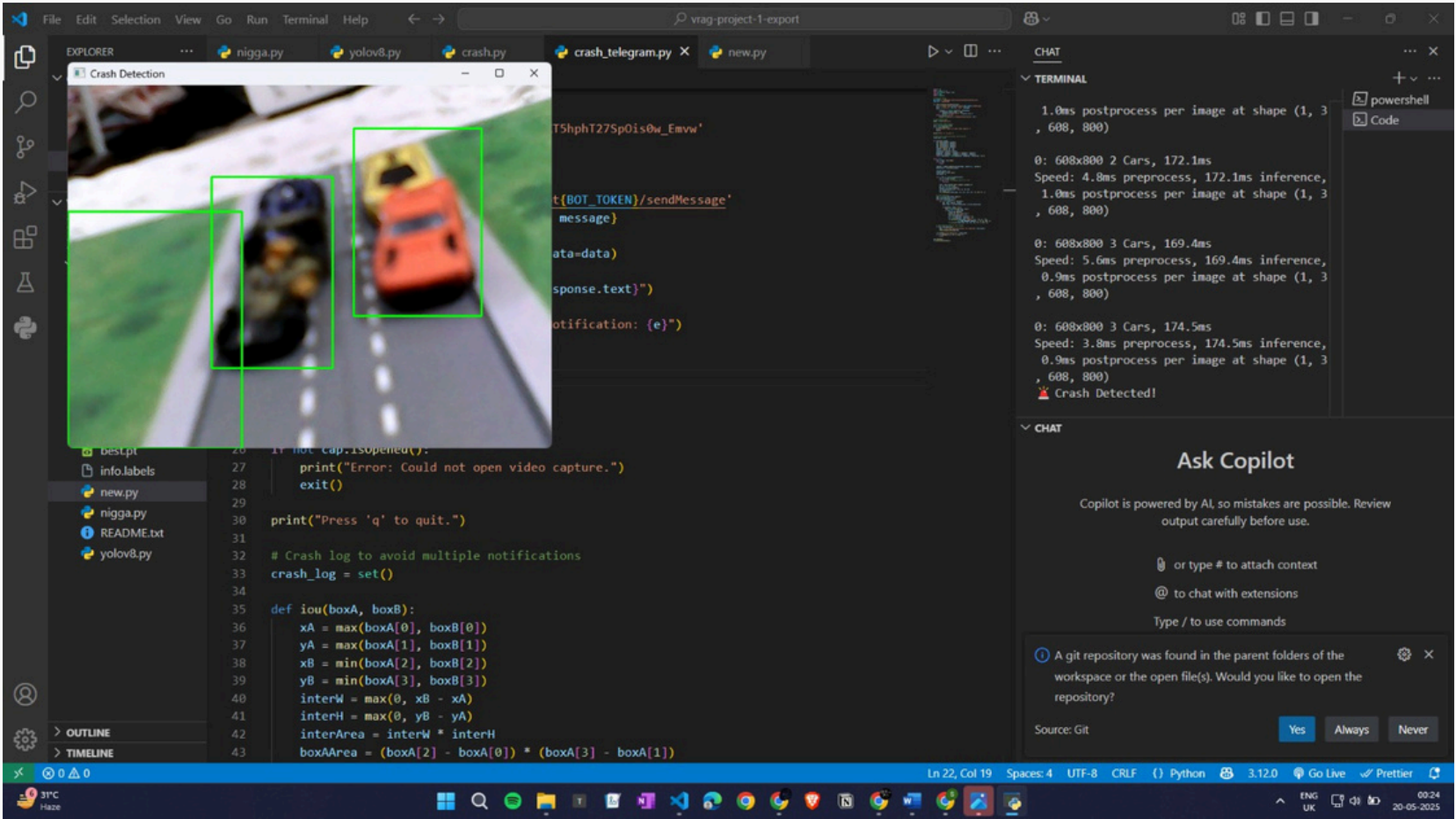
Flow Chart



Results

- Detection FPS: ~4–6 on Jetson Nano
- Telegram alerts sent within 2–3 seconds
- Real-time display of crash zones and bounding boxes
- Stable performance under different angles and lighting

Output



Challenges

- Jetson Nano Limitations: Occasional frame drops under load
- Lighting Conditions: Low light caused false negatives
- Bounding Box Overlap: Very close cars were sometimes merged
- IoU Tuning: Threshold had to be manually adjusted

Learnings

- Hands-on with Embedded AI: Deploying models on Jetson Nano
- YOLOv8 Training Workflow: Dataset preparation, labeling, augmentation, training
- Telegram Bot Integration: Lightweight IoT messaging
- IoU Logic: Real-world application of spatial reasoning
- Teamwork & Problem Solving: Managing time, debugging, and iteration collaboratively

Conclusion

- Successfully built a real-time vehicle detection and crash alerting system.
- Achieved reliable performance on embedded Jetson Nano.
- Demonstrated the feasibility of scalable miniature traffic monitoring systems.

Future Scope

- Add traffic signal detection and control
- Implement multi-camera tracking
- Use cloud-based storage for analytics
- Detect pedestrians, traffic signs, and apply vehicle classification

References



1. Redmon et al., “YOLO: Real-Time Object Detection,” CVPR, 2016.
2. Zhang et al., “YOLO-based Real-Time Vehicle Detection,” IEEE Access, 2020.
3. Ultralytics Docs: <https://docs.ultralytics.com>
4. Telegram Bot API: <https://core.telegram.org/bots/api>

